**Praktikum Algoritma dan Stuktur Data**

**Laporan Jobsheet 5**

**Nama : Yedid Worembai**
**Kelas : 1-G**
**NIM : 254107020254**

### 5.2.2. Verifikasi Hasil Percobaan

```
PROBLEMS 7   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

PS C:\Users\BISMILLAH NAWAITU\PratikumASD>  & 'C:\Program Files\Java\jdk
SMILLAH NAWAITU\AppData\Roaming\Code\User\workspaceStorage\4f6755514bd2d
.FaktorialMain'
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
PS C:\Users\BISMILLAH NAWAITU\PratikumASD>
```

### 5.2.3. Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!

**Jawaban :**

```
int faktorialDC(int n){
    if(n == 1){
        return 1;
    }else{
        int fakto = n * faktorialDC(n - 1);
        return fakto;
    }
}
```

Bagian **if (n == 1)** berfungsi sebagai kondisi dasar (*base case*) dalam rekursi yang digunakan untuk menghentikan pemanggilan fungsi secara berulang. Ketika nilai n = 1, fungsi langsung mengembalikan nilai 1 karena 1! = 1. Tanpa kondisi ini, fungsi akan terus memanggil dirinya sendiri sehingga dapat menyebabkan rekursi tak terbatas (infinite recursion) dan berpotensi menimbulkan stack overflow.

Bagian **else** merupakan tahap rekursif yang menjalankan proses perhitungan utama dalam algoritma Divide and Conquer. Pada bagian ini, nilai faktorial dihitung menggunakan rumus **n! = n × (n−1)!**. Proses tersebut dilakukan dengan cara memecah permasalahan menjadi submasalah yang lebih kecil, yaitu menghitung faktorial dari (n−1) melalui pemanggilan kembali method faktorialDC(), hingga akhirnya mencapai kondisi dasar ketika n = 1.

2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!

**Jawaban :**

```
int faktorialBF(int n){
    int fakto = 1;
     int i = 1;
     while (i <= n) {
    fakto = fakto * i;
    i++;
```

3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !

**Jawaban :**

Perbedaan utama antara pendekatan iteratif (faktorialBF) dan rekursif (faktorialDC) terletak pada efisiensi memori dan mekanisme eksekusinya. Pada metode iteratif, perhitungan dilakukan di dalam satu siklus *loop* yang hanya menggunakan satu variabel penampung, sehingga sangat hemat memori. Sebaliknya, metode rekursif bekerja dengan membagi masalah menjadi sub-masalah yang lebih kecil melalui pemanggilan fungsi itu sendiri. Proses ini memakan lebih banyak memori karena setiap pemanggilan fungsi akan ditumpuk dalam *call stack* sebelum akhirnya dikalikan satu per satu saat mencapai kondisi dasar (*base case*).

4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

**Jawaban :**

Perbedaan mendasar antara kedua metode ini terletak pada paradigma penyelesaian masalahnya. Metode faktorialBF() menerapkan pendekatan iteratif (bottom-up), yang mengakumulasi hasil perkalian secara bertahap mulai dari unit terkecil hingga mencapai nilai **n**. Sebaliknya, faktorialDC() menggunakan pendekatan rekursif (top-down) melalui prinsip *Divide and Conquer*, di mana masalah utama didekomposisi menjadi sub-masalah yang lebih kecil secara berturut-turut hingga mencapai *base case*, sebelum akhirnya hasil tersebut dikombinasikan kembali untuk memperoleh solusi akhir.

### 5.3.2. Verifikasi Hasil Percobaan

```
PROBLEMS 7   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

Masukan nilai basis elemen ke-1: 2
Masukan nilai pangkat elemen ke-1: 3
Masukan nilai basis elemen ke-2: 4
Masukan nilai pangkat elemen ke-2: 5
Masukan nilai basis elemen ke-3: 6
Masukan nilai pangkat elemen ke-3: 7
HASIL PANGKAT DIVIDE AND CONQUER:
HASIL PANGKAT BRUTERORCE:
2^3: 8
4^5: 1024
6^7: 279936
PS C:\Users\BISMILLAH NAWAITU\PratikumASD>
```

### 5.3.3. Pertanyaan

1.Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!

**Jawaban :**

Perbedaan mendasar antara pangkatBF() (Brute Force) dan pangkatDC() (Divide and Conquer) terletak pada mekanisme eksekusi dan tingkat efisiensinya. pangkatBF() menghitung nilai eksponensiasi secara iteratif satu per satu, sehingga sangat hemat memori namun memiliki kompleksitas waktu linear $O(n)$ yang akan berjalan lambat pada pangkat besar. Sebaliknya, pangkatDC() beroperasi secara rekursif dengan membelah nilai eksponen menjadi sub-masalah yang lebih kecil pada setiap langkahnya, menghasilkan kompleksitas waktu logaritmik $O(\log n)$ yang jauh lebih cepat, meskipun membutuhkan sedikit tambahan memori untuk *call stack* rekursif. Kesimpulannya, pangkatBF() memadai untuk perhitungan berskala kecil, sementara pangkatDC() adalah solusi komputasi yang paling optimal untuk angka berskala masif.

2. Apakah tahap combine sudah termasuk dalam kode tersebut?Tunjukkan!

**Jawaban :**

```
 return(pangkatDC(a, n/2)*pangkatDC(a, n/2)*a);
lse{
 return (pangkatDC(a, n/2)*pangkatDC(a, n/2));
```

3. Pada method pangkatBF()terdapat parameter untuk melewatkan nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut

nilai dan pangkat,
apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter?
Apakah bisa jika method tersebut dibuat dengan tanpa parameter?
Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?

**Jawaban :**

1. Penggunaan parameter pada method pangkatBF(int a, int n) tetap relevan dalam konteks fleksibilitas fungsional. Pendekatan ini memungkinkan metode bersifat independen (*stateless*), di mana komputasi nilai dapat dilakukan secara variatif tanpa harus melakukan modifikasi pada *state* atau nilai atribut internal objek tersebut.
2. Penggunaan metode tanpa parameter pada pangkatBF() lebih selaras dengan prinsip enkapsulasi dalam OOP. Karena nilai dan pangkat sudah menjadi atribut internal *class*, metode dapat mengakses data tersebut secara implisit tanpa perlu dikirim ulang.
3. 

```
}int pangkatBF() {
  int hasil = 1;
  for (int i = 0; i < pangkat; i++) {
      hasil = hasil * nilai;
  }
  return hasil;
}
```

4.Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!

**Jawaban :**

perbedaan antara kedua fungsi tersebut terletak pada paradigma komputasi yang diterapkan untuk mencapai solusi akhir. Method pangkatBF() mengadopsi pendekatan iteratif (bottom-up), di mana hasil perpangkatan diperoleh melalui akumulasi perkalian secara linear di dalam sebuah struktur perulangan hingga mencapai eksponen **n**. Sebaliknya, pangkatDC() menerapkan prinsip rekursif (top-down) melalui strategi *Divide and Conquer*. Dalam metode ini, basis eksponen didekomposisi secara logaritmik menjadi sub-masalah yang lebih kecil hingga mencapai *base case*, yang kemudian hasilnya dikombinasikan kembali untuk mereduksi kompleksitas waktu komputasi secara signifikan.

**5.4.2. Verifikasi Hasil Percobaan**

```
PS C:\Users\BISMILLAH NAWAITU\PratikumASD>  & 'C:\Program Files\Jav
-cp' 'C:\Users\BISMILLAH NAWAITU\AppData\Roaming\Code\User\workspac
' 'BruteForceDivideConquer.Minggu5.MainSum'
Masukan jumlah elemen: 5
Masukan Keuntungan ke-1: 10
Masukan Keuntungan ke-2: 20
Masukan Keuntungan ke-3: 30
Masukan Keuntungan ke-4: 40
Masukan Keuntungan ke-5: 50
Total keuntungan menggunakan Bruteforce: 150.0
Total keuntungan menggunakan Devide and conquer:  150.0
PS C:\Users\BISMILLAH NAWAITU\PratikumASD>
```

**5.4.3. Pertanyaan**

1. Kenapa dibutuhkan variable mid pada method TotalDC()?

**Jawaban :**

Dalam algoritma *Divide and Conquer*, tahap Divide (Pecah) bertujuan untuk membagi masalah berskala besar menjadi sub-masalah yang lebih kecil. Variabel mid (indeks tengah) adalah wujud nyata dari tahap pemecahan tersebut di dalam kode.

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

**Jawaban :**

Secara operasional, kedua instruksi tersebut berfungsi untuk mengeksekusi tahap Conquer melalui pemanggilan rekursi guna menyelesaikan sub-masalah secara independen. Variabel lsum bertugas mengkomputasi total nilai elemen *array* pada partisi kiri mulai dari rentang indeks l hingga mid, sedangkan rsum menangani komputasi pada partisi kanan mulai dari rentang indeks mid+1 hingga r. Proses komputasi dari kedua fungsi ini berjalan secara terpisah di dalam tumpukan memori (*stack*) sebelum hasil akhirnya diakumulasikan kembali pada tahap akhir (*Combine*).

3.Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

**Jawaban :**

diperlukan karena ia merupakan representasi langsung dari tahap akhir dalam algoritma *Divide and Conquer*, yaitu tahap Combine (Penggabungan).

4.Apakah base case dari totalDC()?

**Jawaban :**

*Base case* dari totalDC() adalah kondisi if (l == r). Kondisi ini terpenuhi ketika *array* telah terpecah hingga menyisakan satu elemen tunggal. Pada titik ini, rekursi dihentikan dan fungsi langsung mengembalikan nilai elemen tersebut (return arr[l];) untuk diakumulasikan pada tahap *Combine*

5. Tarik Kesimpulan tentang cara kerja totalDC()

**Jawaban :**

Secara sederhana, metode totalDC() bekerja dengan cara membelah tumpukan data (*array*) menjadi dua bagian secara terus-menerus (Divide). Pembelahan ini baru akan berhenti ketika hanya tersisa satu data tunggal, yaitu saat kondisi l == r terpenuhi (*base case*). Setelah data terpecah menjadi unit paling kecil, program mulai memproses potongan kiri dan kanan tersebut secara mandiri (Conquer). Terakhir, hasil dari potongan-potongan tadi dijumlahkan kembali setahap demi setahap dari bawah ke atas (Combine) sampai kita mendapatkan nilai total keseluruhan dari *array* tersebut.

**4.5 Latihan Praktikum**

**Tentukan:**

a.Nilai UTS tertinggi tertinggi menggunakan Divide and Conquer!

**Jawaban :**

```
public int cariMaxDC(int[]arr , int l, int r){
   if (l == r){
     return arr[l];
   }
   int mid = (l + r) / 2;
   int maxKiri = cariMaxDC(arr, l, mid);
   int maxKanan = cariMaxDC(arr, mid + 1, r);
   return Math.max(maxKiri, maxKanan);
}
```

```
public int cariMaxDC(int[]arr , int l, int r){
    if (l == r){
        return arr[l];

    }
    int mid = (l + r) / 2;
    int maxKiri = cariMaxDC(arr, l, mid);
    int maxKanan = cariMaxDC(arr, mid + 1, r);
    return Math.max(maxKiri, maxKanan);
```

b) Nilai UTS terendah menggunakan Divide and Conquer!

**Jawaban :**

```
public int cariMinDC(int[] arr, int l, int r){
  if (l == r){
    return arr[l];
  }
  int mid = (l + r) / 2;
  int minKiri = cariMinDC(arr, l, mid);
  int minKanan = cariMinDC(arr, mid + 1, r);
  return Math.min(minKiri,minKanan);
}
```

```
}
public int cariMinDC(int[] arr, int l, int r){
    if (l == r){
        return arr[l];
    }
    int mid = (l + r) / 2;
    int minKiri = cariMinDC(arr, l, mid);
    int minKanan = cariMinDC(arr, mid + 1, r);
    return Math.min(minKiri,minKanan);

}
```

c) Rata-rata nilai UAS dari semua mahasiswa menggunakan Brute Force!

**Jawaban :**

```
public double hitungRataRataBF(int[] arr){
   double total = 0;
   for (int i = 0; i< arr.length; i++){
     total += arr[i];
   }
   return total / arr.length;
```

}

```
}
public double hitungRataRataBF(int[] arr){
    double total = 0;
    for (int i = 0; i< arr.length; i++){
        total += arr[i];
    }
    return total / arr.length;
}
```

**Hasilnya :**

```
package BruteForceDivideConquer.Minggu5;
public class PraktikumMain {
  public static void main(String[] args) {

    Praktikum p = new Praktikum();

    int[] nilaiUTS = {78, 85, 90, 76, 92, 88, 80, 82};
    int[] nilaiUAS = {82, 88, 87, 79, 95, 85, 83, 84};
    int n = nilaiUTS.length;

    int utsTertinggi = p.cariMaxDC(nilaiUTS, 0, n - 1);
    int utsTerendah = p.cariMinDC(nilaiUTS, 0, n - 1);
    double rataRataUAS = p.hitungRataRataBF(nilaiUAS);

    System.out.println("=== Hasil Latihan Praktikum 4.5 ===");
    System.out.println("a) Nilai UTS tertinggi (Divide & Conquer) : " + utsTertinggi);
    System.out.println("b) Nilai UTS terendah  (Divide & Conquer) : " + utsTerendah);
    System.out.println("c) Rata-rata nilai UAS (Brute Force)     : " + rataRataUAS);
  }
}
```

```
ge\4f6755514bd2dfae235784b6d0647482\redhat.java\jdt_
=== Hasil Latihan Praktikum 4.5 ===
a) Nilai UTS tertinggi (Divide & Conquer) : 92
b) Nilai UTS terendah  (Divide & Conquer) : 76
c) Rata-rata nilai UAS (Brute Force)      : 85.375
PS C:\Users\BISMILLAH NAWAITU\PratikumASD>
```